Sakum Lang

Self-Learning & Self-Healing Engine

How it compiles, learns from failure, and auto-fixes itself

1. What the engine is

The engine is a local, self-hosted agent (tools/sakum_bot.sh) that keeps the Sakum core alive and current. On every pulse (timer, webhook, or websocket frame) it reads the doctrine (SAKUM_LANG.md), its learning rules (learn.md), and its memory (memory.md). It then recompiles the entire assembly core, measures a 'survivability' metric, and rewrites code paths that fail so the same mistake is never repeated.

Survivability is defined as: survive / (survive + mistakes). Every clean compile+run cycle increments the 'survive' counter in memory.md; every failure is written to the mistake ledger (memory.md #mistakes and query_logs/type_1_memory.jsonl) and the offending generated files are rolled back.

The learning loop (learn.md cycle)

- Read doctrine + memory (SAKUM_LANG.md, learn.md, memory.md).
- Check upstream signals (SIMD/WASM/quantum/crypto/memory-safety news) and run the Bramann crawler.
- If a signal maps to a missing capability, draft a REAL, compilable patch (no stubs).
- Recompile EVERY assembly/sakum_*.s with gcc -arch x86_64.
- On compile failure -> log mistake, roll back the cycle's files, relaunch via launchd KeepAlive.
- On success -> append outcome to memory.md and the binary-hash ledger; increment 'survive'.

2. How it works

Inputs the engine lives from

- memory.md - append-only ledger; 'survive:' rolling success counter, patches_applied, last_cycle.
- learn.md   - the hard rules the bot must follow on every pulse.
- assembly/sakum_*.s - the raw x86-64 (and ARM64/RISC-V) core that gets recompiled.
- query_logs/type_1_memory.jsonl - the mistake ledger as binary-hash records.
- docs/asm/spec_*.s - the verified spec corpus (mac build prints FNV1a(spec)=0xC46A785B).

The compile+run gate

The build is driven by the Makefile, which auto-detects host OS/ISA and builds all x86-64 targets with 'gcc -arch x86_64 -include assembly/platform.inc'. Cross-ISA files (arm64, riscv64, arm32) are built only with their native/cross toolchains and are NOT counted as failures on an x86-64 host. A target must both COMPILE and RUN to count toward survivability.

Self-healing control flow

 # pseudocode of the healing loop

```
  on every pulse():
      read SAKUM_LANG.md, learn.md, memory.md
      signals = fetch_updates() + bramann_crawl()
      if signals -> emit real compilable patch (self/patches/patch_<ts>.json)
      ok = recompile_all('assembly/sakum_*.s')   # gcc -arch x86_64
      if not ok:
          ledger.log(mistake)            # memory.md #mistakes + jsonl
          rollback(cycle_files)          # delete generated .s + patch json
          relaunch()                     # launchd KeepAlive
      else:
          memory.survive += 1            # survivability counter up
          ledger.append(outcome)         # binary-hash '#what' note
```

3. Auto-fix in action (real cycle)

On the cycle stamped 1784360162 the engine recompiled all 26 x86-64 targets. Two genuine mistakes were found, diagnosed, and fixed automatically. After the fix: PASS=26, RUN_OK=9, survivability bumped 47 -> 48.

Mistake A - sakum_pipe.s: macro collision

The file redefined SYS_READ etc. as assembler '=' equ's, but platform.inc already does '#define SYS_READ 0x2000003'. The C preprocessor expanded SYS_READ before the assembler saw it, producing the invalid line '0x2000003 = 0x2000000 + 3'. Fix: guard each equ with #ifndef so it only defines when platform.inc has not already.

assembly/sakum_pipe.s

```
# BEFORE (broken):
#ifdef PLAT_MACOS
SYS_READ   = 0x2000000 + 3      ; cpp expands SYS_READ -> '0x2000003 = ...'
#else
SYS_READ   = 0
#endif

# AFTER (fixed):
#ifdef PLAT_MACOS
  #ifndef SYS_READ
    SYS_READ   = 0x2000000 + 3
  #endif
  ...
#else
  #ifndef SYS_READ
    SYS_READ   = 0
  #endif
  ...
#endif
```

Mistake B - sakum_sniff.s: reserved-name + section bug

Two bugs: (1) labels db0..db3 collided with GAS's 'db' (define-byte) directive, so '[rip + db0]' failed with 'invalid base+index expression'. (2) After the RODATA string block there was no TEXT_SECTION switch, so main + all code landed in __cstring, causing a linker relocation error. Fix: renamed db*/ob* to oct_d*/oct_s* and added TEXT_SECTION before main.

assembly/sakum_sniff.s

```
# BEFORE (broken):
db0:      .long 0          ; 'db' is GAS define-byte -> symbol collision
  mov [rip + db0], eax     ; invalid base+index expression
s_io:     .asciz "ioctl"
CDECL(main):               ; code emitted into __cstring -> link error

# AFTER (fixed):
oct_d0:    .long 0         ; renamed away from reserved 'db'
  mov [rip + oct_d0], eax  ; clean RIP-relative store
s_io:     .asciz "ioctl"
TEXT_SECTION               ; <-- switch back to code section
CDECL(main):
```

4. The mistake ledger & survivability

After the cycle, memory.md records both the heal and the metrics:

memory.md (excerpt)

```
survive: 48
last_cycle: 1784360162

## mistakes
mistake 1784360162: target=sakum_sniff reason=symbols db0..db3 collide
   with GAS 'db' directive, and code landed in __cstring (no TEXT_SECTION
   after RODATA) -> invalid base+index + relocation error.
   fix=renamed db*/ob* to oct_d*/oct_s* and added TEXT_SECTION before main.
   status=fixed
mistake 1784360162: target=sakum_pipe reason=SYS_* redefined over
   platform.inc macros -> cpp expanded to '0x2000003 = 0x2000000 + 3'.
   fix=guarded each equ with #ifndef. status=fixed

## learned
learned 1784360162: signal=self_learning_engine patch=engine_cycle
   note=ran engine: 26/26 x86-64 targets OK, 9/9 self-tests OK,
   2 mistakes found+fixed, survivability 47->48.
```

Why it never repeats a mistake

- Failures are logged with target + root cause + the exact fix applied.
- Generated patches are reversible: deleting the patch JSON rolls the cycle back.
- launchd KeepAlive relaunches the bot, so a crash is itself a survivable event.
- The 'survive' counter only rises on a clean compile+run, so metrics stay honest.

Run it yourself

shell

```
# build + run the whole core (x86-64 host)
make                 # build all targets
make test            # build + run library self-tests

# or run the engine cycle directly
for f in assembly/sakum_*.s; do
  gcc -arch x86_64 -x assembler-with-cpp -include assembly/platform.inc \
      "$f" -o /tmp/sakum_build/$(basename $f .s) || echo FAIL $f
done
```